

第五讲 漏洞利用技术

一、shellcode 概述

统一用 shellcode 这个专用术语来通称缓冲区溢出攻击中植入进程的代码。之后讨论的 shellcode 是植入进程的代码，而不是狭义上的仅仅用来获得 shell 的代码。

植入代码之前我们需要做大量的调试工作。

例如，弄清楚程序有几个输入点，这些输入会当做哪个函数的参数读入到内存的哪一个区域，哪一个输入会造成栈溢出等等。

调试后还要计算函数返回地址和缓冲区的偏移并且淹没来使 shellcode 得到执行。

这个代码的植入过程就是漏洞利用，即 exploit。

二、定位 shellcode

见 PPT

三、缓冲区的组织

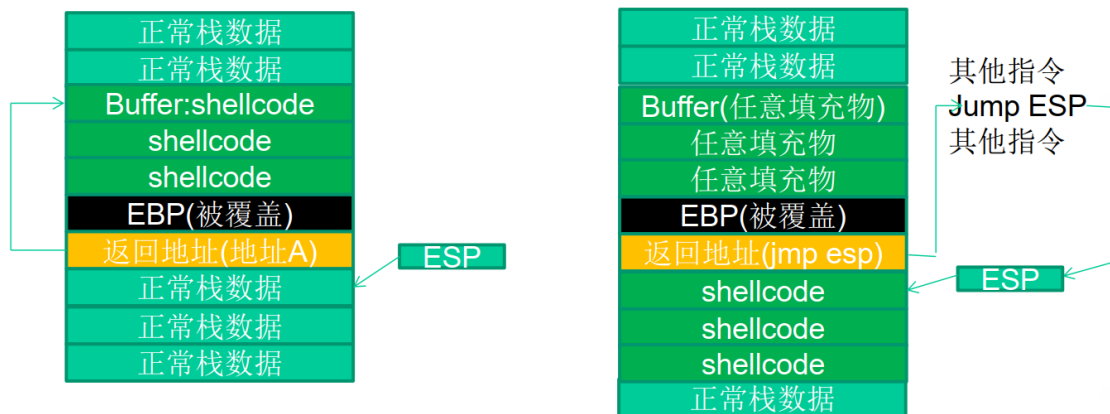
1、缓冲区的组成

如果选用 `jmp esp` 作为定位 shellcode 的跳板，那么在函数返回后要根据缓冲区大小、所需 shellcode 长短等实际情况灵活地布置缓冲区。送入缓冲区的数据可分为以下几种：

(1) 填充物：可以是任何值，但是一般用 NOP 指令对应的 `0x90` 来进行填充，这样只要能跳进填充区，处理器最终也能顺序执行到 shellcode。

(2) 淹没返回地址的数据：可以是跳转指令的地址，shellcode 的起始地址，或者近似的 shellcode 地址（跳转进 NOP 填充区）

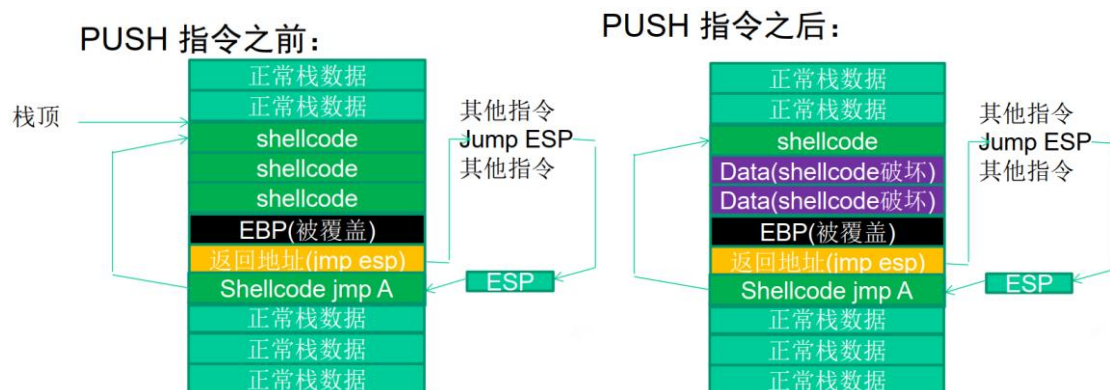
(3) shellcode：可执行的机器代码。



返回shellcode静态地址	返回jmp esp跳板地址
合理利用缓冲区，使攻击串总长度减小	能够适应动态变化的shellcode地址
对程序破坏小，比较稳定，不会大范围破坏前栈帧	可能会破坏前栈帧的数据，无法修复寄存器的值

另一种组织方式：在返回地址后再淹没一点，在那里布置一个 shellcode header，指向一大片真正的 shellcode 中。

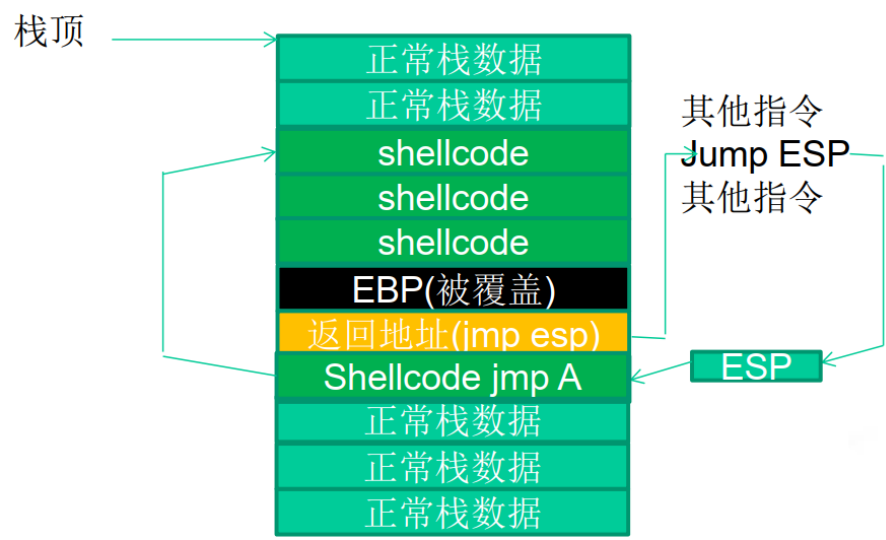
但将 shellcode 布置到缓冲区可能会发生如下的问题：



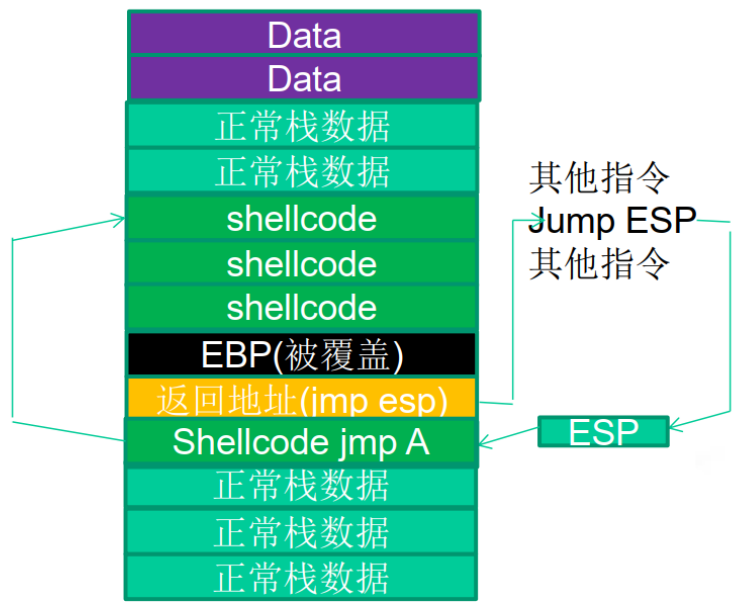
2、抬高栈顶保护 shellcode

为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的 shellcode。

为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的 shellcode。PUSH操作之前：

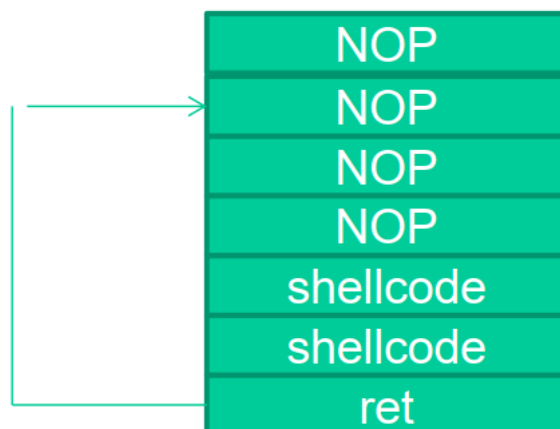


为了防止上述情况的发生，我们可以采用抬高栈顶来保护我们的 shellcode。PUSH操作之后：

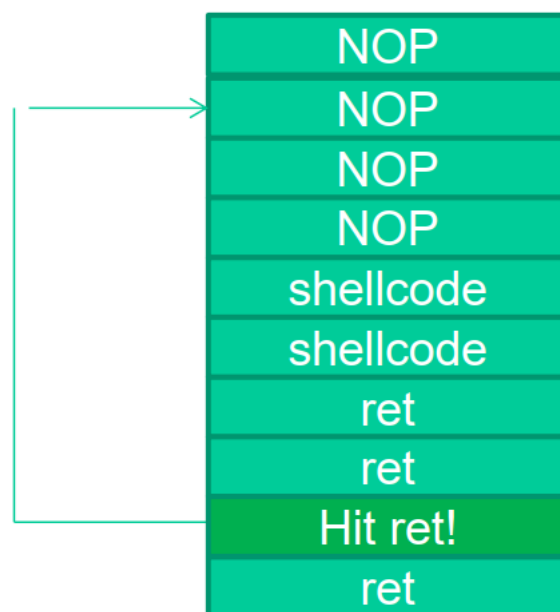


3、函数返回地址移位

在一些情况下，返回地址距离缓冲区的偏移量是不确定的，我们可以增加 NOP 来增加“靶子面积”。



如果函数返回地址的偏移按双字(DWORD)不定，可以用一片连续的跳转指令来覆盖函数的返回地址。



四、shellcode 编码技术

在很多漏洞利用场景中，shellcode 的内容将会受到限制。

首先，所有的字符串函数都会对 NULL 字节进行限制。

其次，有些函数还会要求 shellcode 必须为可见字符的 ASCII 或 Unicode 的值。

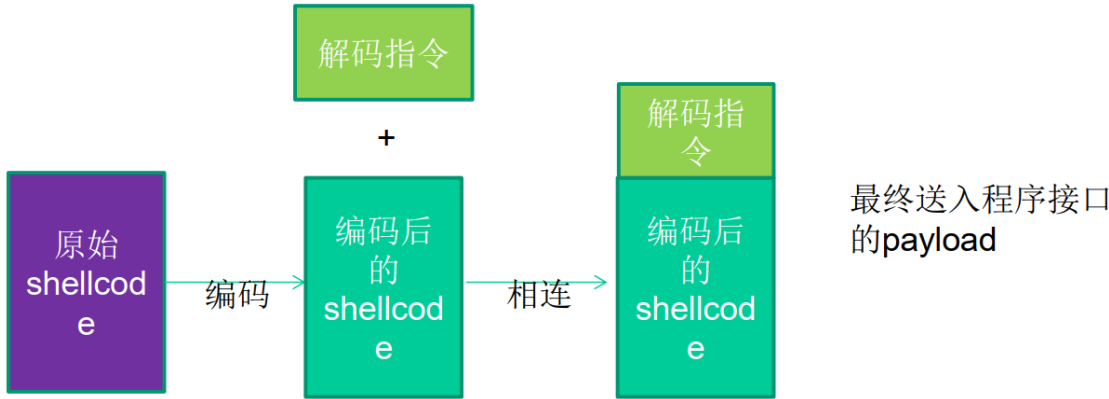
最后，在进行网络攻击时，基于特征的系统往往会对常见的 shellcode 进行拦截。

那么我们就只能通过给 shellcode 进行一下编码来使 shellcode “蒙混过关” 后再行动。

我们先在 shellcode 的前面加上十几个字节的解码程序放在 shellcode 开始执行的地方。

当 exploit 成功时，解码程序首先运行，然后将内存中的变形的 shellcode 解码成原来样子，从而顺利执行。

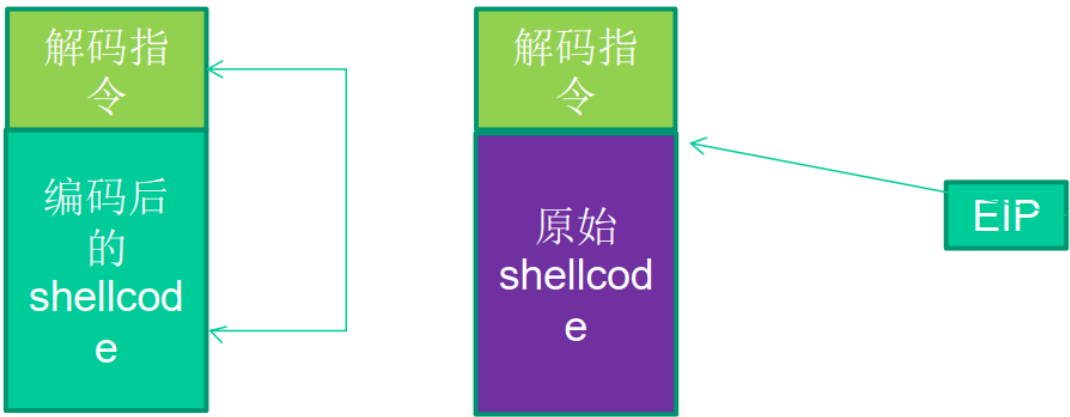
shellcode 编码示意图：



Shellcode 解码示意图：

首先由解码指令
还原shellcode

解码完毕之后继续
执行真正的shellcode



五、Metasploit 简介

漏洞利用技术中一些相对独立的过程。

- 1) 触发漏洞：缓冲区有多大，第几个字节可以淹没返回地址，用什么样的方法植入代码？
- 2) 选取 shellcode: 执行什么样的 shellcode 决定了漏洞利用的性质。例如，是作为安全测试而弹出的一个消息框，还是用于入侵的端口绑定、木马上传等。
- 3) 重要参数的设定：目标主机的 IP 地址、bindshell 中需要绑定的端口号、消息框所显示的内容、跳转指令的地址等经常需要在 shellcode 中进行修改。
- 4) 选用编码、解码算法：我们曾经介绍过，实际应用中的 shellcode 往往需要经过编码（加密）才能安全地送入特定的缓冲区；执行时，位于 shellcode 顶部的若干条解码指令会首先还原出原始的 shellcode，然后执行。

MSF 包括的模块以及每个模块都包含哪些东西，以及各个模块的作用：

模块类型	目前MSF4.0包含的数量	说明
Exploit	754	包含已公布漏洞的触发信息，如返回地址偏移量等
Auxiliary	418	MSF额外的插件程序，如网络欺骗工具、DOS工具、Sniffer工具等
Payload	227	就是我们说的shellcode
Encoder	27	编码算法
nop	8	“准nop”填充数据生成器。“准nop”是指不影响shellcode执行的指令。除了最经典的0x90(nop)之外，如果EBX的值不影响shellcode的执行，那么0x43(inc ebx)就是“准nop”填充数据。